Contents

1 System Architecture

1.1 AI-Based Intrusion Detection Framework for Next-Generation Networks

Detailed Multi-Layer Architecture Design
Last Updated: December 22, 2025

1.2 Table of Contents

1.3 1. Overview

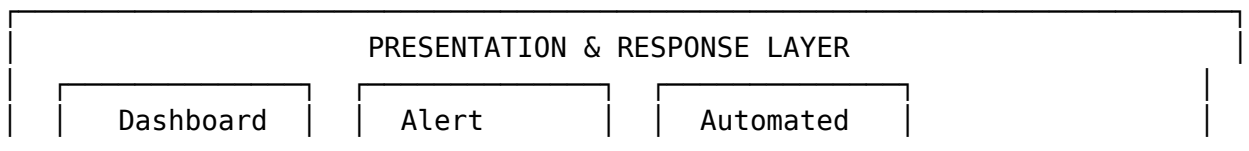
The proposed AI-IDS framework employs a modular, layered architecture designed for scalability, flexibility, and performance. The system integrates multiple AI/ML techniques, distributed computing paradigms, and explainability mechanisms to provide comprehensive intrusion detection across diverse next-generation network environments.

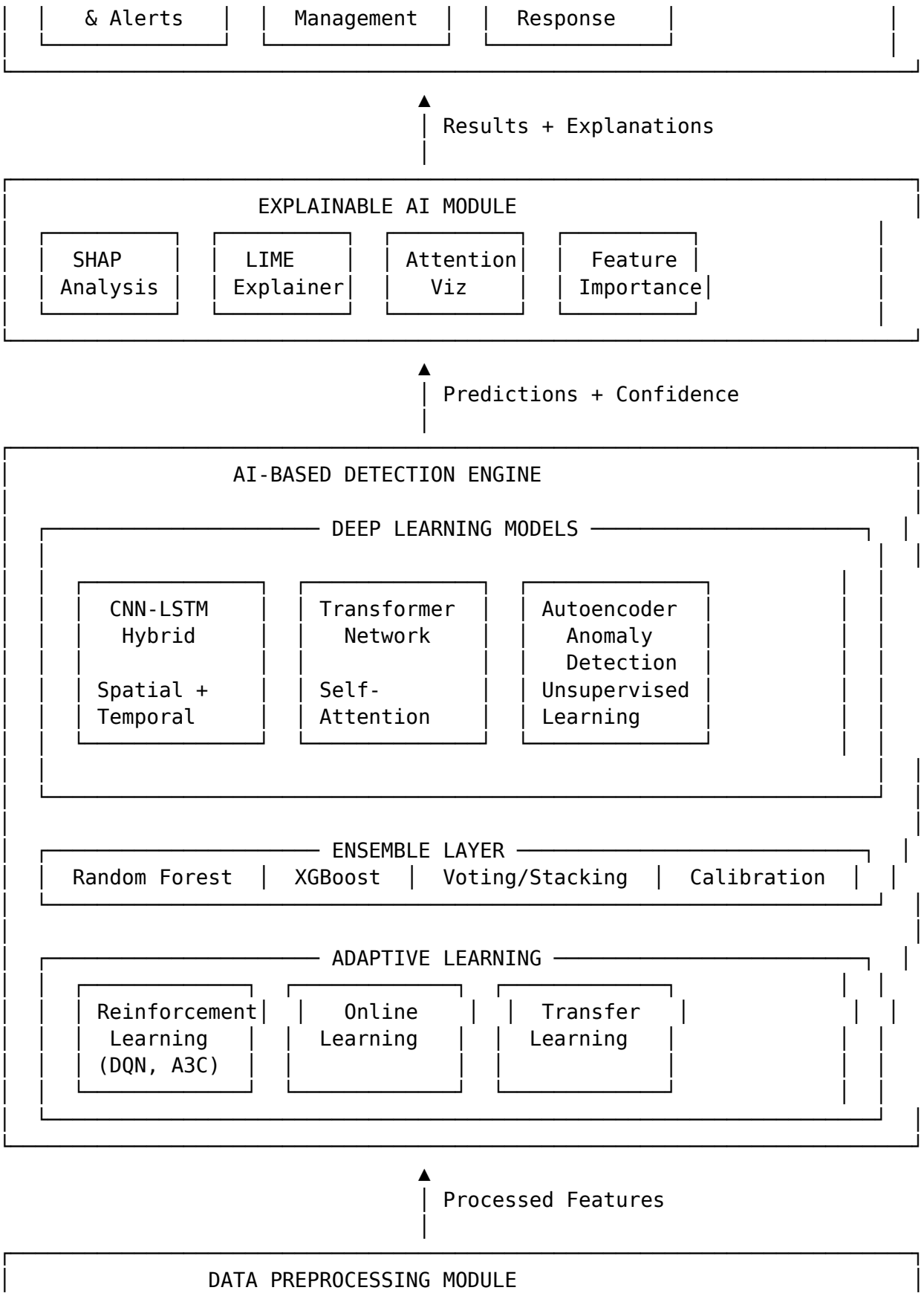
Key Design Principles: - **Modularity:** Independent components with well-defined interfaces - **Scalability:** Horizontal and vertical scaling capabilities - **Extensibility:** Easy integration of new models and data sources - **Real-Time Performance:** <100ms detection latency for critical scenarios - **Privacy-Preserving:** Federated learning and differential privacy support - **Explainability:** Built-in XAI for transparent decision-making

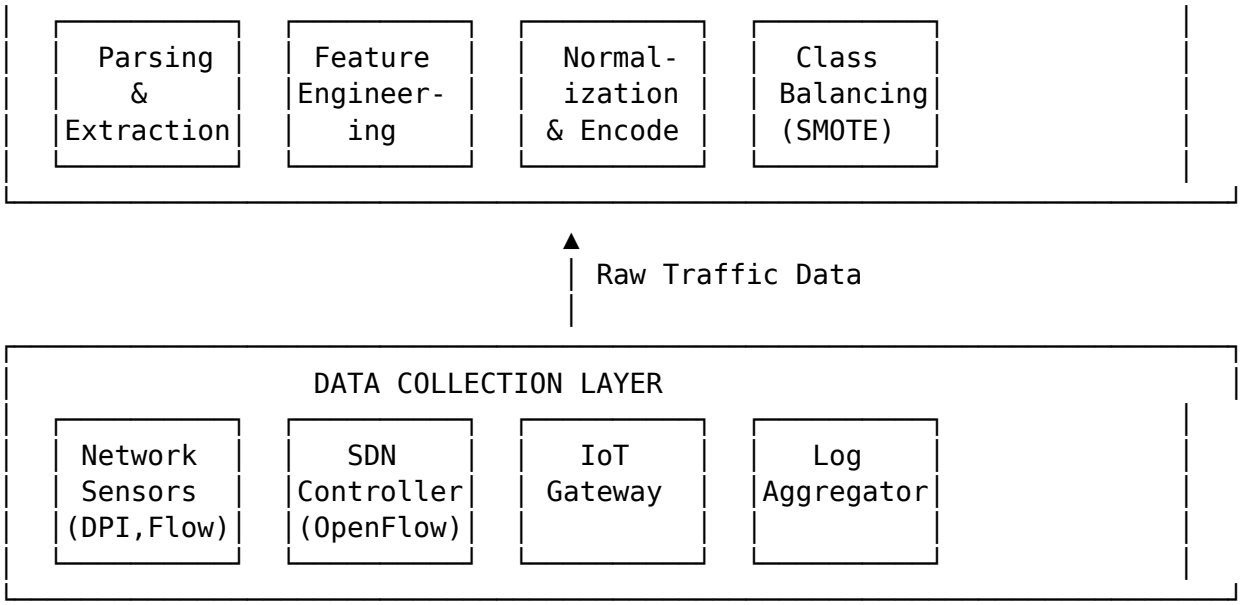
---

1.4 2. Multi-Layer Architecture

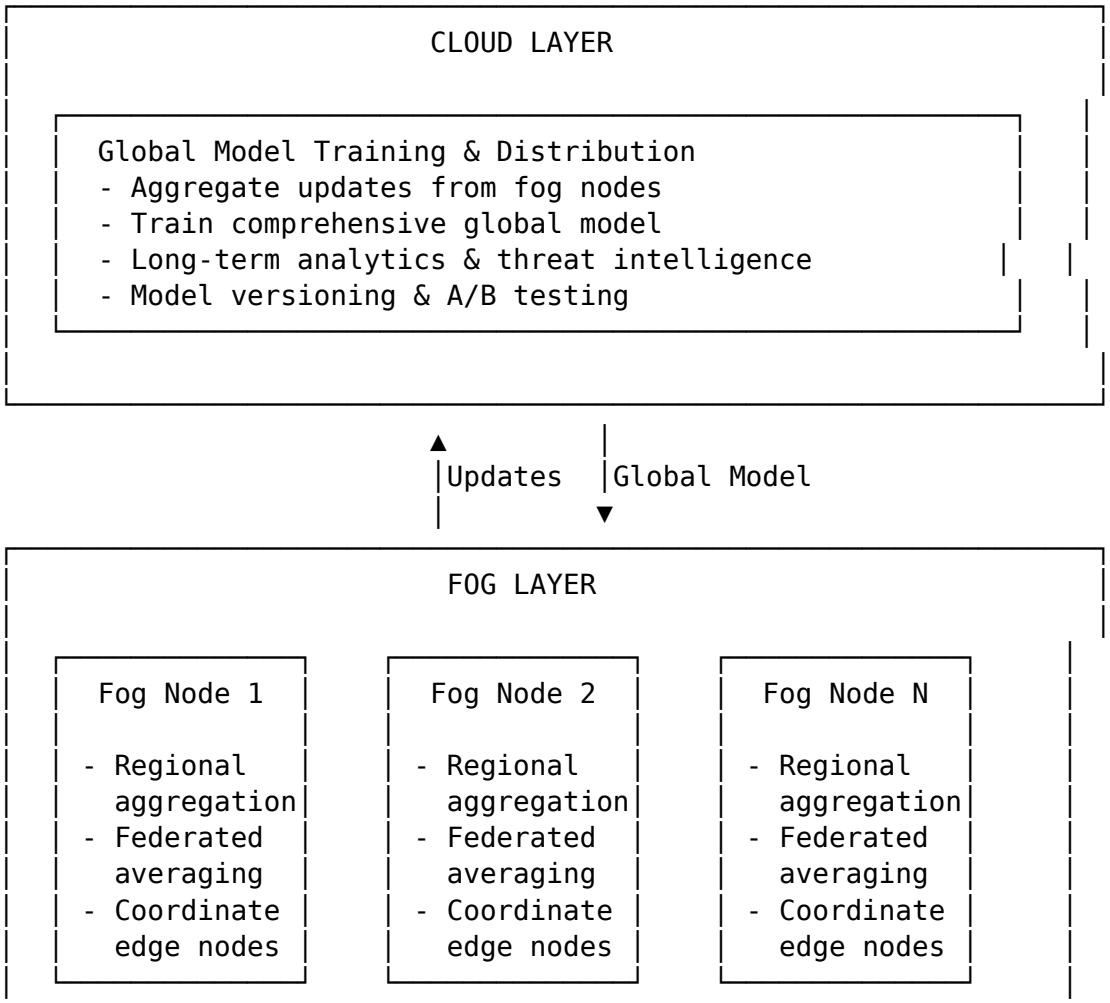
1.4.1 2.1 High-Level Architecture Diagram

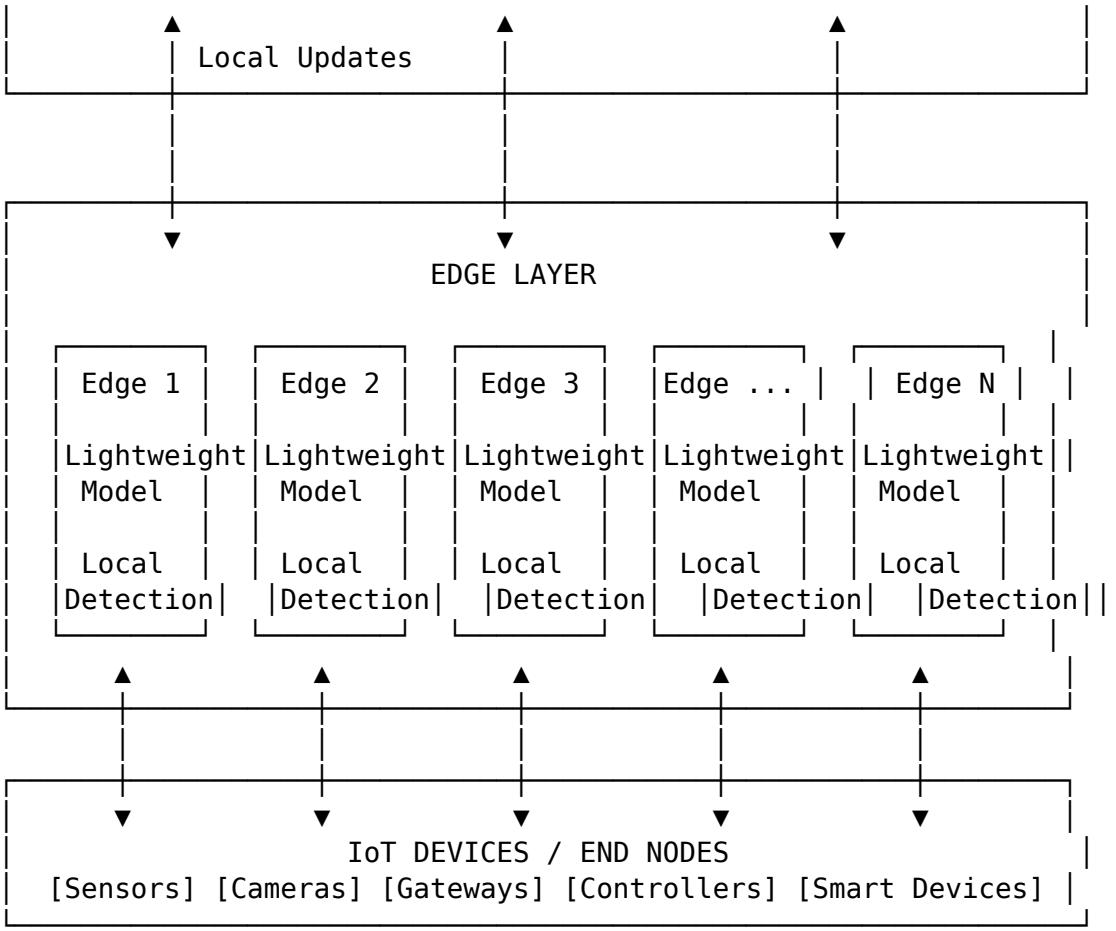



& Alerts
Management
Response
Results + Explanations
EXPLAINABLE AI MODULE
SHAP Analysis
LIME Explainer
Attention Viz
Feature Importance
Predictions + Confidence
AI-BASED DETECTION ENGINE
DEEP LEARNING MODELS
CNN-LSTM Hybrid
Spatial + Temporal
Transformer Network
Self-Attention
Autoencoder Anomaly Detection
Unsupervised Learning
ENSEMBLE LAYER
Random Forest
XGBoost
Voting/Stacking
Calibration
ADAPTIVE LEARNING
Reinforcement Learning (DQN, A3C)
Online Learning
Transfer Learning
Processed Features
DATA PREPROCESSING MODULE



1.4.2 2.2 Distributed Edge-Fog-Cloud Architecture





1.5 3. Data Collection Layer

1.5.1 3.1 Network Traffic Sensors

Deep Packet Inspection (DPI): - Full packet capture and analysis - Protocol parsing (HTTP, DNS, TLS, etc.) - Payload inspection (within legal/privacy bounds) - Tools: tcpdump, Wireshark/tshark, Zeek

Flow-Based Monitoring: - NetFlow/IPFIX/sFlow collectors - 5-tuple identification: (src_ip, dst_ip, src_port, dst_port, protocol) - Flow statistics: duration, packet count, byte count, flags - Tools: nfdump, SiLK, Softflowd

Placement: - Network edge (ingress/egress points) - Core switches/routers - Before/after firewalls - DMZ boundaries

1.5.2 3.2 SDN Controller Integration

OpenFlow Statistics Collection:

```
SDN Controller (ONOS/OpenDaylight)
          │
```

```
├─> Flow Tables (per-switch)
│   - Flow rules, matches, actions
│   - Packet/byte counters
│   - Duration, priority
│
├─> Topology Information
│   - Switch connectivity
│   - Host locations
│   - Link utilization
│
└─> Control Plane Events
    - Packet-In messages
    - Flow modifications
    - Port status changes
```

REST API Integration: - Poll controller APIs every 1-5 seconds - Subscribe to event notifications (webhooks) - Extract: flow stats, topology, events

1.5.3 3.3 IoT Gateway Monitoring

Protocol-Specific Collectors: - MQTT broker monitoring (pub/sub patterns, message rates) - CoAP proxy logging (requests, responses) - Zigbee/Z-Wave coordinator data - LoRaWAN network server statistics

Device Behavior Tracking: - Communication patterns (frequency, peers) - Data volumes (upload/download) - Command sequences - Timing analysis

1.5.4 3.4 Log Aggregation

System Logs: - Syslog from servers, network devices, applications - Authentication logs (login attempts, failures) - Application logs (errors, warnings, access)

Security Logs: - Firewall logs (permit/deny decisions) - IPS/IDS alerts (Snort, Suricata) - Antivirus detections - SIEM events

Aggregation Tools: - ELK Stack (Elasticsearch, Logstash, Kibana) - Splunk - Graylog - Fluentd

---

1.6 4. Data Preprocessing Module

1.6.1 4.1 Traffic Parsing & Feature Extraction

Packet-Level Features (41 features from NSL-KDD): - Duration, protocol_type, service, flag - src_bytes, dst_bytes, land, wrong_fragment - urgent, hot, num_failed_logins, logged_in - num_compromised, root_shell, su_attempted - num_root, num_file_creations, num_shells - num_access_files, num_outbound_cmds - is_host_login, is_guest_login

Flow-Level Features (80+ features from CICIDS2017): - Flow duration, fwd/bwd packets, fwd/bwd bytes - Packet length (mean, std, min, max) - IAT (Inter-Arrival Time) statistics - Flags count (FIN, SYN, RST, PSH, ACK, URG) - Window size statistics - Down/up ratio, avg packet size

Statistical Features: - Mean, median, std, variance - Percentiles (25th, 50th, 75th) - Skewness, kurtosis - Time-windowed aggregations

Behavioral Features: - Connection rate (connections per second) - Unique hosts contacted - Port diversity - Protocol distribution - Temporal patterns (time of day, day of week)

1.6.2 4.2 Feature Engineering Pipeline

```
# Pseudocode for Feature Engineering
def extract_features(raw_traffic):
    features = {}

    # Basic features from packet headers
    features['duration'] = flow.end_time - flow.start_time
    features['protocol'] = flow.protocol
    features['src_port'] = flow.src_port
    features['dst_port'] = flow.dst_port

    # Statistical features
    features['pkt_len_mean'] = mean(flow.packet_lengths)
    features['pkt_len_std'] = std(flow.packet_lengths)
    features['pkt_len_max'] = max(flow.packet_lengths)
    features['pkt_len_min'] = min(flow.packet_lengths)

    # Inter-arrival time features
    iat = compute_inter_arrival_times(flow.timestamps)
    features['iat_mean'] = mean(iat)
    features['iat_std'] = std(iat)

    # Flag counts
    features['fin_flag_count'] = count_flags(flow, 'FIN')
    features['syn_flag_count'] = count_flags(flow, 'SYN')

    # Payload features (if available)
    if flow.has_payload:
        features['payload_entropy'] = compute_entropy(flow.payload)

    return features
```

1.6.3 4.3 Normalization & Encoding

Numerical Feature Normalization:

```
# Min-Max Scaling (0-1 range)
X_normalized = (X - X.min()) / (X.max() - X.min())

# Z-Score Standardization (mean=0, std=1)
X_standardized = (X - X.mean()) / X.std()
```

Categorical Encoding:

```
# One-Hot Encoding for low-cardinality (< 10 categories)
protocol_encoded = pd.get_dummies(df['protocol'])

# Label Encoding for ordinal or high-cardinality
service_encoded = LabelEncoder().fit_transform(df['service'])

# Embedding for very high-cardinality (learned during training)
embedding_layer = Embedding(input_dim=vocab_size, output_dim=16)
```

1.6.4 4.4 Class Balancing

Problem: Imbalanced datasets (90%+ normal traffic)

Solutions:

SMOTE (Synthetic Minority Over-sampling):

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='minority', k_neighbors=5)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

ADASYN (Adaptive Synthetic Sampling):

```
from imblearn.over_sampling import ADASYN

adasyn = ADASYN(sampling_strategy='minority', n_neighbors=5)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)
```

Class Weights:

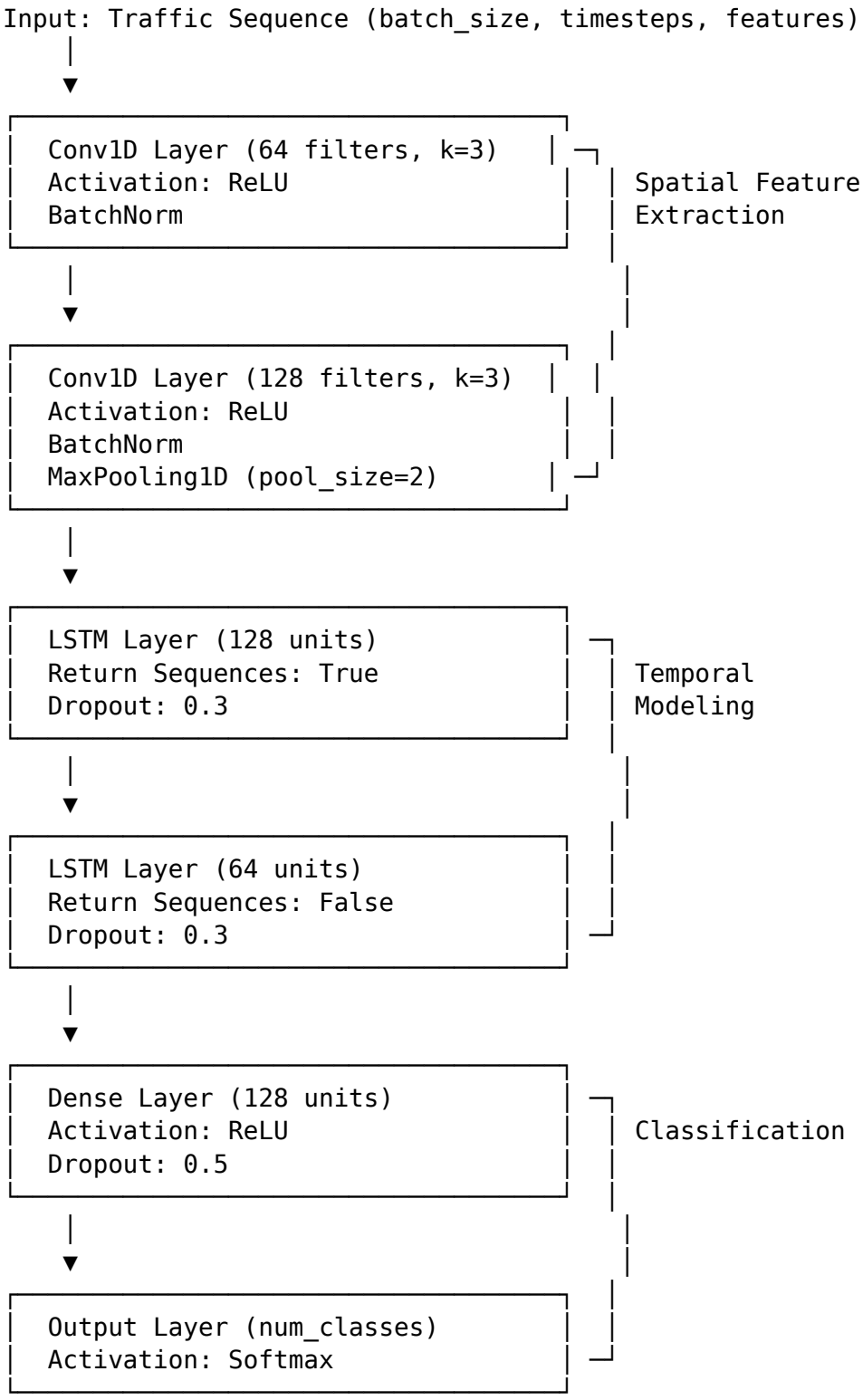
```
# Compute class weights inversely proportional to frequency
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train),
                                     y=y_train)

# Use in model training
model.fit(X_train, y_train, class_weight=class_weights)
```

1.7 5. AI-Based Detection Engine

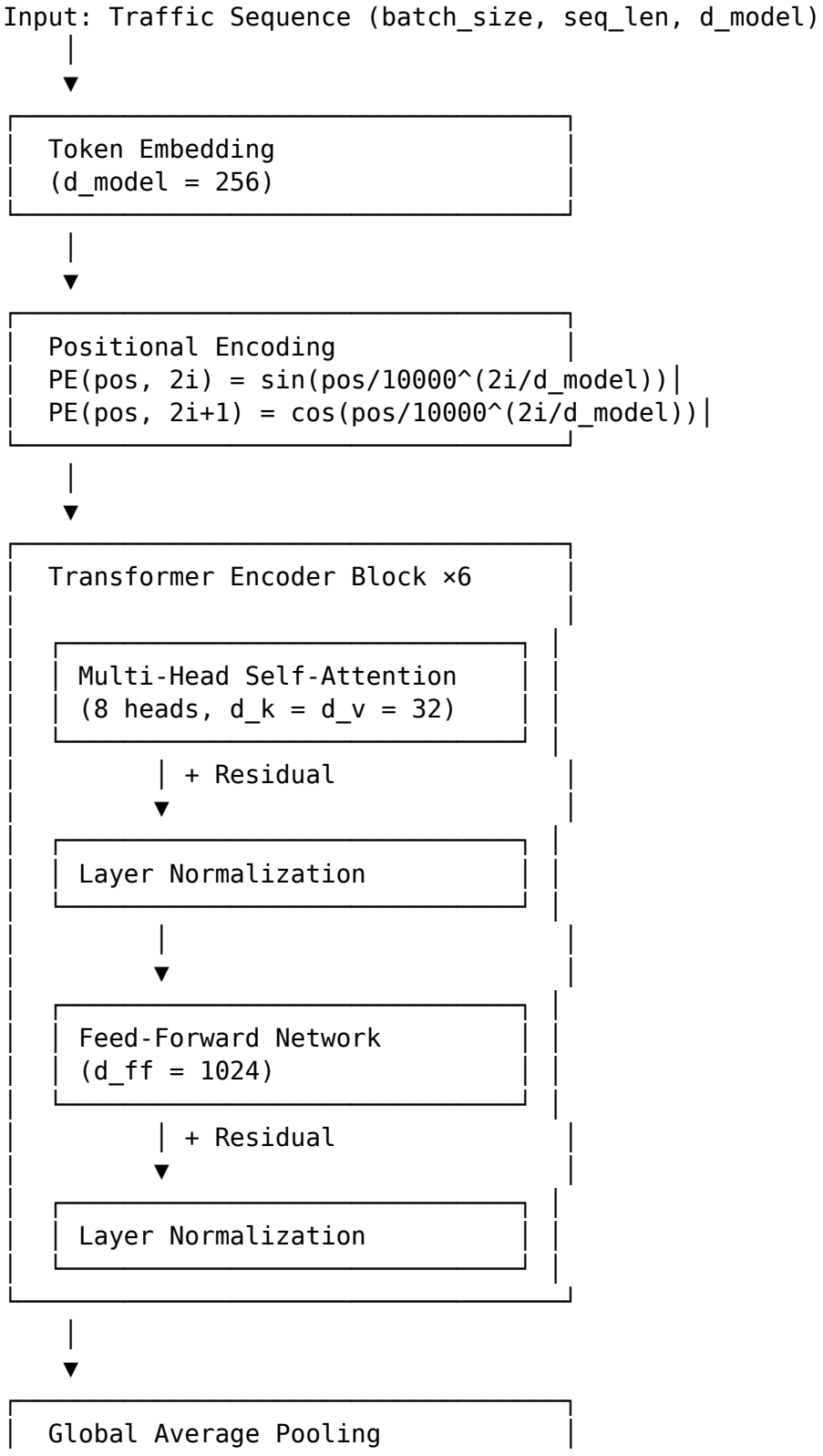
1.7.1 5.1 CNN-LSTM Hybrid Model

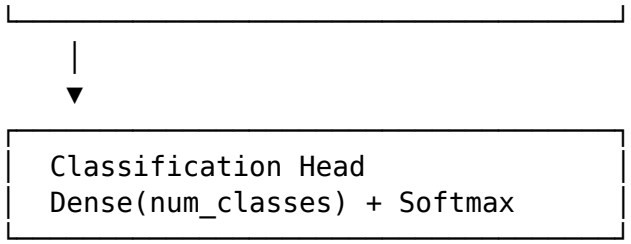
Architecture:



1.7.2 5.2 Transformer Network

Architecture:





1.7.3 5.3 Autoencoder for Anomaly Detection

Architecture:

```
Encoder:
Input (n_features) → Dense(128, relu) → Dense(64, relu)
    → Dense(32, relu) → Latent (16)

Decoder:
Latent (16) → Dense(32, relu) → Dense(64, relu)
    → Dense(128, relu) → Output (n_features)

Loss: Mean Squared Error between Input and Output

Anomaly Detection:
reconstruction_error = mse(input, output)
if reconstruction_error > threshold:
    classify as anomaly
else:
    classify as normal
```

1.7.4 5.4 Ensemble Layer

Voting Ensemble:

```
# Hard Voting (Majority)
predictions = [model1.predict(X), model2.predict(X), model3.predict(X)]
final_prediction = mode(predictions)  # Most frequent class

# Soft Voting (Weighted Average of Probabilities)
probas = [model1.predict_proba(X), model2.predict_proba(X),
          model3.predict_proba(X)]
avg_proba = np.mean(probas, axis=0)
final_prediction = np.argmax(avg_proba, axis=1)
```

Stacking Ensemble:

```
# Level 0: Base Models
base_models = [cnn_lstm, transformer, rf, xgboost]

# Level 1: Meta-Model
meta_features = []
```

```
for model in base_models:
    predictions = model.predict_proba(X_train)
    meta_features.append(predictions)

meta_X = np.column_stack(meta_features)
meta_model = LogisticRegression()
meta_model.fit(meta_X, y_train)

# Prediction
meta_X_test = np.column_stack([m.predict_proba(X_test) for m in base_models])
final_prediction = meta_model.predict(meta_X_test)
```

1.7.5 5.5 Adaptive Learning

Reinforcement Learning (DQN):

```
# State: Current network state (traffic features, system metrics)
# Action: Response action (alert, block, rate_limit, ignore)
# Reward: +1 correct detection, -0.5 false positive, -2 missed attack

class DQNAgent:
    def __init__(self, state_size, action_size):
        self.q_network = build_q_network(state_size, action_size)
        self.target_network = build_q_network(state_size, action_size)
        self.memory = ReplayBuffer(maxlen=10000)

    def act(self, state, epsilon):
        if random() < epsilon:
            return random_action()  # Explore
        else:
            q_values = self.q_network.predict(state)
            return argmax(q_values)  # Exploit

    def train(self, batch_size=32):
        batch = self.memory.sample(batch_size)
        for state, action, reward, next_state, done in batch:
            target = reward
            if not done:
                target += gamma * max(self.target_network.predict(next_state))

            q_values = self.q_network.predict(state)
            q_values[action] = target
            self.q_network.fit(state, q_values)
```

Online Learning:

```
# Incremental learning with streaming data
def online_learning_update(model, new_data, new_labels):
    # Partial fit for models supporting incremental learning
```

```
    model.partial_fit(new_data, new_labels)

    # For deep learning: mini-batch gradient descent
    model.fit(new_data, new_labels, epochs=1, batch_size=32)

    # Exponential moving average for concept drift adaptation
    model.weights = alpha * model.weights + (1-alpha) * new_weights
```

1.8 6. Explainable AI Module

1.8.1 6.1 SHAP Integration

```
import shap

# Train tree-based model for SHAP
model = XGBClassifier()
model.fit(X_train, y_train)

# Compute SHAP values
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Global feature importance
shap.summary_plot(shap_values, X_test)

# Local explanation for single prediction
shap.force_plot(explainer.expected_value, shap_values[i], X_test[i])
```

1.8.2 6.2 LIME Integration

```
from lime import lime_tabular

# Create LIME explainer
explainer = lime_tabular.LimeTabularExplainer(
    X_train,
    feature_names=feature_names,
    class_names=['Normal', 'Attack'],
    mode='classification'
)

# Explain single prediction
explanation = explainer.explain_instance(
    X_test[i],
    model.predict_proba,
    num_features=10
)
```

```
# Visualize
explanation.show_in_notebook()
```

1.8.3 6.3 Attention Visualization

```
# For Transformer or LSTM with attention
attention_weights = model.get_layer('attention').get_weights()

# Heatmap of attention across time steps
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.heatmap(attention_weights[0], cmap='viridis',
            xticklabels=time_steps, yticklabels=features)
plt.title('Attention Heatmap: Which time steps are important?')
plt.xlabel('Time Steps')
plt.ylabel('Features')
plt.show()
```

1.9 7. Decision & Response Layer

1.9.1 7.1 Alert Generation

```
def generate_alert(prediction, confidence, features, explanation):
    alert = {
        'timestamp': datetime.now(),
        'severity': determine_severity(prediction),
        'attack_type': prediction,
        'confidence': confidence,
        'source_ip': features['src_ip'],
        'destination_ip': features['dst_ip'],
        'protocol': features['protocol'],
        'explanation': explanation,
        'recommended_action': suggest_action(prediction, confidence)
    }
    return alert
```

1.9.2 7.2 Automated Response

```
def automated_response(alert):
    if alert['confidence'] > 0.95 and alert['severity'] == 'critical':
        # High-confidence critical alert: automatic blocking
        block_ip(alert['source_ip'])
        log_action('BLOCKED', alert)

    elif alert['confidence'] > 0.8:
```

```
        # Medium-high confidence: rate limiting
        rate_limit_ip(alert['source_ip'], limit=100)  # 100 req/sec
        log_action('RATE_LIMITED', alert)

    else:
        # Lower confidence: log for human review
        queue_for_review(alert)
        log_action('QUEUED', alert)
```

1.9.3 7.3 Human-in-the-Loop

```
# Dashboard API for analyst interaction
def analyst_feedback(alert_id, decision):
    alert = get_alert(alert_id)

    if decision == 'true_positive':
        # Confirmed attack: strengthen detection
        update_model_with_feedback(alert, label='attack', weight=1.5)

    elif decision == 'false_positive':
        # False alarm: adjust to reduce similar FPs
        update_model_with_feedback(alert, label='normal', weight=1.5)

    # Retrain model periodically with feedback
    if accumulated_feedback > threshold:
        retrain_model_with_feedback()
```

1.10 8. Distributed Architecture for Edge Computing

1.10.1 8.1 Federated Learning Protocol

```
# Server-side aggregation (FedAvg)
def federated_averaging(local_models):
    global_model = initialize_model()

    # Average weights across all local models
    for layer in global_model.layers:
        weights = [model.get_layer(layer.name).get_weights()
                   for model in local_models]
        avg_weights = np.mean(weights, axis=0)
        global_model.get_layer(layer.name).set_weights(avg_weights)

    return global_model

# Client-side training
def local_training(global_model, local_data):
    local_model = clone_model(global_model)
```

```
    local_model.fit(local_data, epochs=5, batch_size=32)
    return local_model
```

1.10.2 8.2 Model Compression for Edge

Quantization:

```
import tensorflow as tf

# Post-training quantization (8-bit)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

# Model size reduction: ~4× smaller
```

Pruning:

```
import tensorflow_model_optimization as tfmot

# Magnitude-based pruning (remove 50% of weights)
prune_low_magnitude = tfmot.sparsity.keras.prune_low_magnitude

model = prune_low_magnitude(model,
    pruning_schedule=tfmot.sparsity.keras.PolynomialDecay(
        initial_sparsity=0.0, final_sparsity=0.5,
        begin_step=0, end_step=1000
    ))

model.fit(X_train, y_train, epochs=10)
```

Knowledge Distillation:

```
# Teacher: Large, accurate model
# Student: Small, efficient model

def distillation_loss(y_true, y_pred_student, y_pred_teacher, T=3.0):
    # Soft targets from teacher
    soft_targets = tf.nn.softmax(y_pred_teacher / T)

    # Student learns from soft targets
    distill_loss = tf.keras.losses.categorical_crossentropy(
        soft_targets, tf.nn.softmax(y_pred_student / T)
    )

    # Also learn from hard labels
    student_loss = tf.keras.losses.categorical_crossentropy(
        y_true, y_pred_student
    )
```

```
    return alpha * distill_loss + (1 - alpha) * student_loss
```

1.11 9. Performance Optimization

1.11.1 9.1 Latency Reduction

Batch Processing:

```
# Process multiple samples together for efficiency
batch_size = 128
latency_per_sample = batch_processing_time / batch_size
```

Model Optimization: - Use INT8 quantization (4× faster inference) - TensorRT optimization for NVIDIA GPUs - ONNX Runtime for cross-platform deployment - Model pruning (remove 50-70% of weights)

Caching:

```
from functools import lru_cache

@lru_cache(maxsize=1000)
def feature_extraction(traffic_hash):
    # Cache extracted features for repeated flows
    return extract_features(traffic)
```

1.11.2 9.2 Scalability

Horizontal Scaling: - Deploy multiple detection instances - Load balancer distributes traffic - Shared nothing architecture

Parallel Processing:

```
from multiprocessing import Pool

def process_traffic_parallel(traffic_batches):
    with Pool(processes=8) as pool:
        results = pool.map(detect_intrusion, traffic_batches)
    return results
```

GPU Acceleration:

```
# Use GPU for deep learning inference
with tf.device('/GPU:0'):
    predictions = model.predict(X_test, batch_size=1024)
```

1.12 10. Implementation Technologies

1.12.1 10.1 Core Framework Stack

Deep Learning: - TensorFlow 2.13+ / PyTorch 2.0+ - Keras (high-level API) - TensorFlow Lite (edge deployment) - ONNX (model interoperability)

Classical ML: - Scikit-learn 1.3+ - XGBoost 2.0+ - LightGBM 4.0+

Data Processing: - Pandas 2.0+ (data manipulation) - NumPy 1.25+ (numerical computing) - PySpark 3.4+ (distributed processing)

1.12.2 10.2 Network & SDN Tools

SDN Controllers: - ONOS 2.7+ (carrier-grade) - OpenDaylight Argon+ (modular) - Ryu (Python-based, lightweight)

Network Simulation: - Mininet (SDN testbed) - NS-3 (network simulator) - OMNET++ (discrete event simulation)

1.12.3 10.3 XAI & Visualization

Explainability: - SHAP 0.42+ - LIME 0.2+ - Captum (PyTorch XAI)

Visualization: - Matplotlib 3.7+ - Seaborn 0.12+ - Plotly 5.15+ (interactive) - Grafana (dashboards)

1.12.4 10.4 Distributed Computing

Federated Learning: - TensorFlow Federated 0.57+ - PySyft 0.8+ (privacy-preserving ML) - Flower 1.5+ (FL framework)

Containerization: - Docker 24+ (containers) - Kubernetes 1.28+ (orchestration)

1.12.5 10.5 Monitoring & Logging

Metrics: - Prometheus (time-series metrics) - Grafana (dashboards)

Logging: - ELK Stack (Elasticsearch, Logstash, Kibana) - Fluentd (log aggregation)

1.13 Conclusion

This multi-layer architecture provides a comprehensive, scalable, and flexible framework for AI-based intrusion detection in next-generation networks. The modular design allows for independent development, testing, and deployment of components while maintaining system coherence. The distributed edge-fog-cloud architecture addresses scalability and privacy requirements, while the integration of explainable AI ensures transparency and trust in automated security decisions.

Key Advantages: - Modularity enables independent component updates - Distributed architecture scales to millions of devices - Hybrid AI models achieve superior detection accuracy - Explainability builds trust with security analysts - Adaptive learning handles evolving threats - Performance optimizations meet real-time requirements (<100ms)

Next Steps: Refer to `algorithms.md` for detailed algorithm specifications and `evaluation_metrics.md` for comprehensive evaluation methodology.